HW1: Complete Arithmetic Logic Unit Design

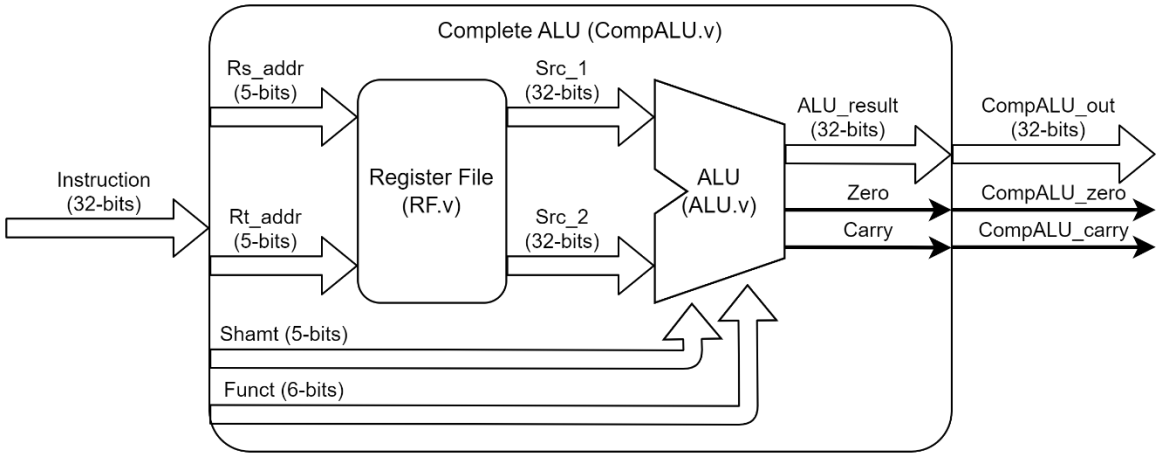
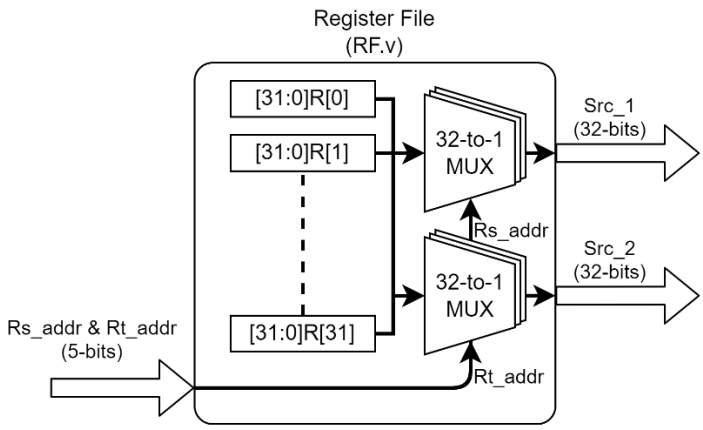


Figure 1: Path of Complete ALU

Implement a 32-bits Complete ALU module, which consists of a Register File module and an ALU module. The Register File module is composed of 32 read-only registers with a width of 32-bits. The ALU module is a 32-bits wide arithmetic logic unit.

32-bits Read Only Register File



<u>I/O Interface</u>

```
module RF (
   input [4:0] Rs_addr,
   input [4:0] Rt_addr,
   output [31:0] Src_1,
   output [31:0] Src_2
);
```

Figure 2: Path of Register File

Implement a register file with 32 registers having a width of 32-bits which are addressed by 5-bits. This module only performs the read operation with two outputs marked as "Rs_data" and "Rt_data".

Note: Your registers must be declared as a 2D array with the exact name **R**. The testbench will directly access and load data into **R**. Any modification to the register name may cause the testbench to fail.

32-bits Arithmetic Logic Unit

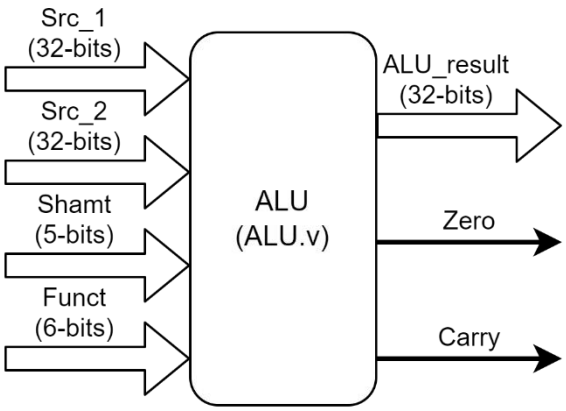


Figure 3: I/O Interface of ALU

I/O Interface

```
module ALU (
  input [31:0] Src_1,
  input [31:0] Src_2,
  input [4:0] Shamt,
  input [5:0] Funct,
  output [31:0] ALU_result,
  output Zero,
  output Carry
);
```

Implement an arithmetic logic unit, which performs related operations based on the register file and then outputs a result. The input of this module contains two 32-bits wide data bus from the register file, a 5-bits shift amount control bus, and a 6-bits function code control bus from the given instructions. The output is a 32-bits wide data bus to output the operation result, and two 1-bit flags, namely "Zero" and "Carry". The Zero flag is used to indicate whether the operation result is zero, and the Carry flag is used for the carry/borrow flag of the result.

Instruction	Example	Meaning	Funct code
Add unsigned	addu Result, Src_1, Src_2	Result = Src_1 + Src_2	100001
Subtract unsigned	subu Result, Src_1, Src_2	Result = Src_1 − Src_2	100011
Nor	nor Result, Src_1, Src_2	Result = ~(Src_1 | Src_2)	100111
Shift right logical	srl Result, Src_2, Shamt	Result = Src_2 ≫ Shamt	000010
Shift left logical	sll Result, Src_2, Shamt	Result = Src_2 ≪ Shamt	000000

Note: The value required for the shift action is stored in "Shamt".

32-bits Complete ALU

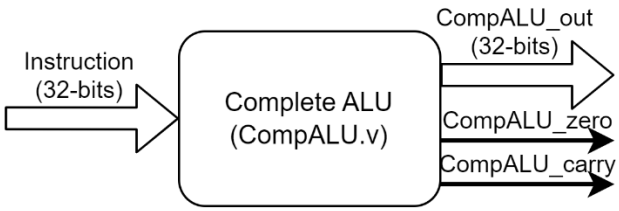


Figure 4: I/O Interface of Complete ALU

<u>I/O Interface</u>

```
module CompALU (
   input [31:0] Instruction,
   output [31:0] CompALU_out,
   output CompALU_zero,
   output CompALU_carry
);
```

Implement 32-bits complete ALU by combing the register file and the arithmetic logic unit. The input of the module is a 32-bits instruction, and the output of this module is a 32-bits result, Zero flag and Carry flag. The instruction format is as follows:

OP Code	Source Register Address	Target Register Address	Destination Register Address	Shamt	Funct Code
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

OP Code: Indicate the command to be executed, this homework is set as “000000”.

Source Register Address: Indicate the address of the source register to be accessed.

Target Register Address: Indicate the address of the target register to be accessed.

Destination Register Address: The addressed register used to store the execution result. It can be set to zero in this homework.

Shamt: Indicate the number of "Shift" actions.

Funct Code: Indicate the command to be executed.

Instance Name

```
46      RF Register_File (
47          // Inputs
48          // Outputs
49      );
```

Figure 5: **RF** Instantiation in **CompALU** Module

Note: The instance of the **RF** module in the **CompALU** module must be named **Register_File** (shown as Figure 5). The testbench will directly load data into **Register_File.R**. Failing to follow this naming requirement may cause the testbench to fail.

<u>Examples:</u>

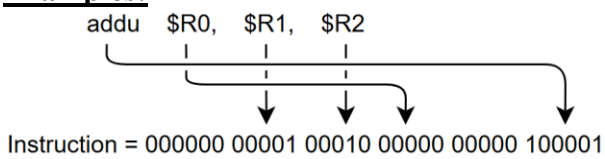


Figure 6: Example of Add Unsigned Instruction

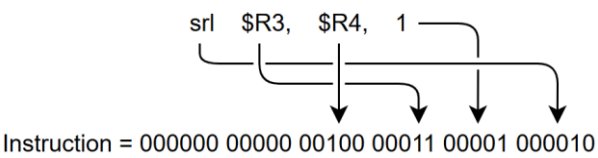


Figure 7: Example of Shift Right Logical Instruction

Design constraints:

1. The use of any **Look-Up-Table realization** is strictly **prohibited** in this homework.
2. Designers must strictly follow the specified **naming conventions** of all design **modules** and corresponding **I/O interfaces**.

Failure to meet the above requirements may result in point deductions.

Behavioral Simulation

1. tb_RF

The testbench "**tb_RF**" initializes the register according to "**RF_in.dat**", and then sequentially sends the address to Rs_addr and Rt_addr. Its output waveform (in hexadecimal) is similar to the figure below. You need to show the screenshot and explain the results in your report. Make sure that you need to output your results to "**RF_out.dat**".

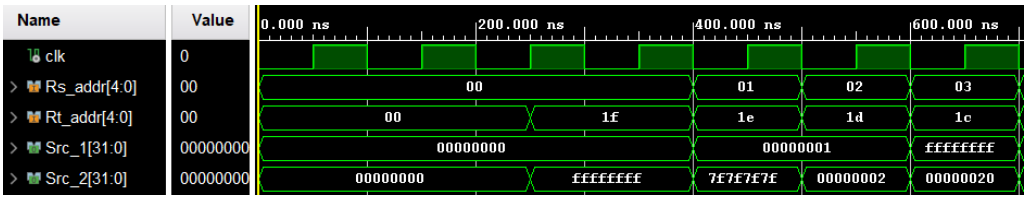


Figure 8: Waveform of **tb_RF**

2. tb_ALU

The testbench "**tb_ALU**" initializes all inputs to zero, and then executes commands in "**ALU_in.dat**" one by one. Its output waveform (in hexadecimal) is similar to the figure below. Please show a screenshot and explain the results in your report. "**ALU_in.dat**" only provides an example for the "addu" operation. You should add other commands and explain the results in the report. Make sure that you need to output your results to "**ALU_out.dat**".

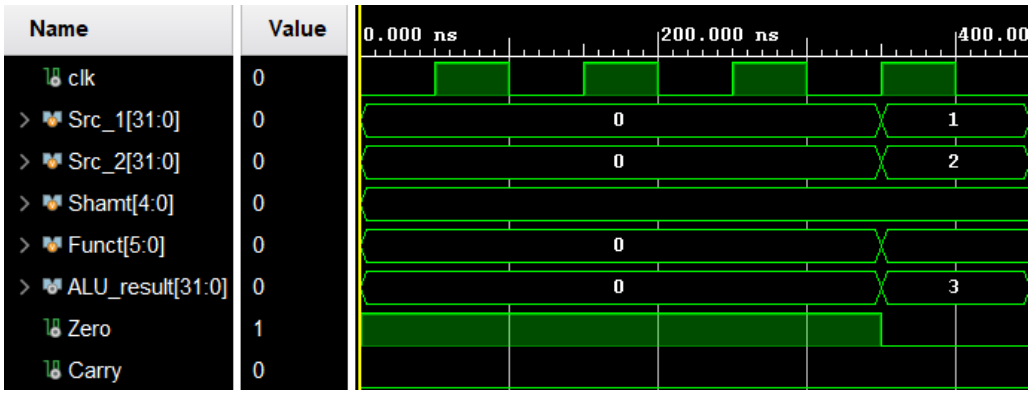


Figure 9: Waveform of **tb_ALU**

3. tb_CompALU

The testbench "**tb_CompALU**" initializes all the input to zero and initialized the register file. Then, the testbench executes the commands in "**CompALU_in.dat**" one by one. Its output waveform (in hexadecimal) is similar to the figure below. Please show a screenshot and explain the results in your report. "**CompALU_in.dat**" only provides one example. You can add other test commands and describe them in the report. Make sure that you need to output your results to "**CompALU_out.dat**".

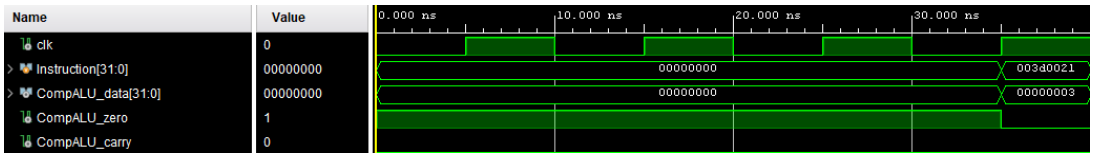


Figure 10: Waveform of **tb_CompALU**

Note: The template testbench is provided on Moodle. You are allowed to modify the testbench for debugging purposes. However, you must ensure that the original template testbench can run successfully without modification. **Your designs will be evaluated using the original template testbench**.

Grading:

a・ (32 points) RF: Each correctly indicated index(address) of Rs_data and Rt_data is worth 1 point. All 32 register indices will be tested.
b・ (20 points) ALU: Points will be awarded based on the correctness and completeness of the ALU functionality.
c・ (20 points) CompALU: Points will be awarded based on the correctness and completeness of the ALU functionality.
d・ (28 points) Report.
e・ **Follow the below specified naming rules and file format requirements.** Points may be deducted if the submission does not comply with the required rules and formats.
f・ **No plagiarism**.

Note: We will test your modules with another generated test vectors.

Submission

Files required for HW1:

- BYYYDDXXX.zip

```
BYYYDDXXX.zip
|
├── RTL/          # Verilog source files
|   ├── RF.v
|   ├── ALU.v
|   └── CompALU.v
|
└── PDF/          # Report files
    └── BYYYDDXXX.pdf
```

※ Convert the report to PDF and name it the student ID - "BYYYDDXXX.pdf".

※ Note: Please make sure RTL folder and PDF folder are directly in the zipped file, not all wrapped in one folder.

Report structure and points:

1. Cover
2. Design Process (12 pts)
   Descriptions, flow charts and architectures of design process for each module.
3. Behavioral Simulation (12 pts)
   Screenshots and explanations of the test results (including waveforms) for each module in behavioral simulation.
4. Conclusion (4 pts)
   Conclusion and insight on this homework.

Report Page Limitation:

The report should be within 10 pages (excluding the cover page).

Deadline: 2026-03-19 12:00 on Moodle

Appendix: **Cover Example**

Computer Organization, Spring 2026

Homework 1:

Complete Arithmetic Logic Unit Design

Lecturer: 陳雅淑教授

Department: Electrical Engineering

Student ID: B11207XXX

Name: 林 X X

Appendix: Testbench description

Ex: tb_CompALU.v

```
// Configuration
`define PERIOD                  10.0
`define TERMINATE_CLOCK_CYCLE   10000.0
`define RST_CLOCK_CYLCE         2.0

`define REGISTER_SIZE           32  // bit width
`define MAX_REGISTER            32  // index

`define DATA_FILE               "../../../../../TESTBED/test_patterns/RF_in.dat"
`define INPUT_FILE              "../../../../../TESTBED/test_patterns/CompALU_in.dat"
`define OUTPUT_FILE             "../../../../../TESTBED/test_patterns/CompALU_out.dat"

module tb_CompALU;

    // Inputs
    reg [31:0] Instruction;

    // Outputs
    wire [31:0] CompALU_data;
    wire CompALU_zero;
    wire CompALU_carry;

    wire glbl_GSR = glbl.GSR;
    // Clock signal
    wire clk;
    clk_gen U_clk_gen (
        .glbl_GSR(glbl_GSR  ),
        .clk     (clk       ),
        .rst     (          ),
        .rst_n   (          )
    );

    // Testbench variables
    reg [`REGISTER_SIZE-1:0] register [0:`MAX_REGISTER-1];
    reg [31:0] read_data;
    integer input_file;
    integer output_file;
    integer i;

    // Instantiate the Unit Under Test (UUT)
    CompALU UUT(
        // Inputs
        .Instruction(Instruction),
        // Outputs
        .CompALU_out(CompALU_data),
        .CompALU_zero(CompALU_zero),
        .CompALU_carry(CompALU_carry)
    );
```

Please modify your file path

```
    initial begin
        // Initialize inputs
        // Format: OpCode_Src1addr_Src2addr_RESERVED_shamt_funct
        Instruction = 32'b000000_00000_00000_00000_00000_000000;

        // Initialize testbench files
        $readmemh(`DATA_FILE, register);
        // Initialize testbench files
        input_file  = $fopen(`INPUT_FILE, "r");
        if (input_file === 0) begin
            $display("Cannot open input file. Check your directory.");
            $finish;
        end
        output_file = $fopen(`OUTPUT_FILE);
        if (output_file === 0) begin
            $display("Cannot open output file. Check your directory.");
            $finish;
        end

        // Initialize internal register
        for (i = 0; i < `MAX_REGISTER; i = i + 1) begin
            UUT.Register_File.R[i] = register[i];
        end

        // Delay
        repeat(3) @(posedge clk);

        // Start testing
        while (!$feof(input_file)) begin
            $fscanf(input_file, "%b\n", read_data);
            @(posedge clk); // Wait clock
            Instruction = read_data;
            @(negedge clk); // Wait clock
            $display("Instruction:%b", read_data);
            $display("CompALU_data:%d, Z:%b, C:%b", CompALU_data, CompALU_zero, CompALU_carry);
            $fdisplay(output_file, "%b,%b,%b", CompALU_data, CompALU_zero, CompALU_carry);
        end

        // Close output file for safety
        $fclose(output_file);

        // Stop the simulation
        $finish;
    end
endmodule
```

```
126 module clk_gen (
127     input      glbl_GSR ,
128     output reg clk      ,
129     output reg rst      ,
130     output reg rst_n
131 );
132     always #(`PERIOD / 2.0) clk = ~clk;
133
134     initial begin
135         clk = 1'b0;
136         wait (glbl_GSR === 1'b1);
137         wait (glbl_GSR === 1'b0);
138         @(posedge clk);
139         rst = 1'b0; rst_n = 1'b1; #(                    0.5  * `PERIOD);
140         rst = 1'b1; rst_n = 1'b0; #((`RST_CLOCK_CYLCE - 0.5) * `PERIOD);
141         rst = 1'b0; rst_n = 1'b1; #(  `TERMINATE_CLOCK_CYCLE * `PERIOD);
142         // Stop the simulation
143         $finish;
144     end
145 endmodule
```

Line	Description
29~38	Verilog compiler directives used for configuration. Please modify the **DATA_FILE**, **INPUT_FILE** and **OUTPUT_FILE** to indicate the **correct absolute path** for your input and output files.
53~58	Generate oscillating clocks.
98	Initialization for registers. Please name your instance name as **Register_File** for your **RF** circuit in **CompALU** module.
107	After the positive edge of the clk signal occurs, the program can continue to be executed.
109	After the negative edge of the clk signal occurs, the program can continue to be executed.
119	Interrupts the simulation to end the simulation.